

Knowledge Distillation: Compressing Neural Networks for Efficient Inference

A Study on CIFAR-10 Image Classification

Ahmed Mahfooz Ali Khan

November 3, 2025

Contents

1	Introduction	3
1.1	The Core Idea	3
1.2	Objectives	3
2	Background and Theory	3
2.1	Traditional Neural Network Training	3
2.2	Soft Labels and Temperature Scaling	4
2.3	The Knowledge Distillation Loss	4
3	Implementation	5
3.1	Dataset: CIFAR-10	5
3.2	Model Architectures	5
3.2.1	Teacher Model	5
3.2.2	Student Model	5
3.3	Distillation Framework	6
3.4	Training Configuration	6
4	Results	7
4.1	Training Progress	7
4.1.1	Teacher Model Training	7
4.1.2	Student Model Training (with KD)	7
4.2	Final Test Performance	7
4.3	Loss Component Analysis	7
5	Discussion	8
5.1	Why Did the Student Outperform the Teacher?	8
5.2	Role of Temperature	8
5.3	Balancing Hard and Soft Losses	8
6	Conclusion	8
6.1	Future Work	9
A	Knowledge Distillation Workflow	9

Abstract

Knowledge Distillation (KD) is a model compression technique that transfers knowledge from a large, complex “teacher” model to a smaller, efficient “student” model. This report presents an implementation of knowledge distillation for image classification on the CIFAR-10 dataset using TensorFlow/Keras. We demonstrate how a student model with fewer parameters can achieve competitive performance by learning from the soft probability distributions produced by the teacher model. Our experiments show that the distilled student model achieved 50.58% test accuracy, surpassing the teacher’s 48.64% accuracy, demonstrating the effectiveness of knowledge transfer.

1 Introduction

Deep neural networks have achieved remarkable success in computer vision tasks, but their computational requirements often make deployment on resource-constrained devices challenging. Knowledge Distillation, introduced by Hinton et al. (2015), offers an elegant solution by enabling smaller models to learn from larger ones.

1.1 The Core Idea

The fundamental insight behind knowledge distillation is simple yet powerful: when a teacher model makes predictions, the probability distribution it produces contains more information than just the correct class label. For example, if a model classifies an image as “cat” with 80% probability, but also assigns 15% to “dog” and 5% to “tiger,” these “soft labels” reveal that the image has features similar to dogs and tigers.

A student model trained on these soft probability distributions learns not just *what* the correct answer is, but *why* certain classes are more similar to each other—capturing the “dark knowledge” embedded in the teacher’s predictions.

1.2 Objectives

This study aims to:

1. Implement a knowledge distillation framework using TensorFlow/Keras
2. Train a teacher model on CIFAR-10 and transfer its knowledge to a student model
3. Compare the performance of the distilled student against the teacher
4. Analyze the components of the distillation loss function

2 Background and Theory

2.1 Traditional Neural Network Training

In conventional supervised learning, a neural network is trained to minimize the cross-entropy loss between its predictions and the ground-truth labels (hard labels):

$$\mathcal{L}_{CE} = - \sum_{i=1}^N \sum_{c=1}^C y_{i,c} \log(p_{i,c}) \quad (1)$$

where $y_{i,c}$ is 1 if sample i belongs to class c (and 0 otherwise), $p_{i,c}$ is the predicted probability for class c , and C is the number of classes.

2.2 Soft Labels and Temperature Scaling

The key innovation in knowledge distillation is the use of “soft labels”—the full probability distribution output by the teacher model, rather than just the hard class label.

To make these distributions more informative, we apply **temperature scaling** to the logits (pre-softmax outputs):

$$p_i = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)} \quad (2)$$

where z_i are the logits, T is the temperature parameter, and p_i is the softened probability.

Effect of Temperature:

- $T = 1$: Standard softmax (sharp distribution)
- $T > 1$: Softer distribution, revealing more information about class relationships
- $T \rightarrow \infty$: Uniform distribution

Higher temperatures “soften” the probability distribution, making the relationships between classes more visible to the student.

2.3 The Knowledge Distillation Loss

The total loss for training the student model combines two components:

$$\mathcal{L}_{KD} = (1 - \alpha) \cdot \mathcal{L}_{CE}^{hard} + \alpha \cdot T^2 \cdot \mathcal{L}_{KL}^{soft} \quad (3)$$

where:

- $\mathcal{L}_{CE}^{hard}$: Cross-entropy loss with true labels (hard loss)
- $\mathcal{L}_{KL}^{soft}$: KL divergence between teacher and student soft distributions
- α : Balancing parameter (0 to 1)
- T^2 : Scaling factor to match gradient magnitudes

The KL divergence is computed as:

$$\mathcal{L}_{KL} = \sum_{c=1}^C p_c^{teacher} \log \left(\frac{p_c^{teacher}}{p_c^{student}} \right) \quad (4)$$

3 Implementation

3.1 Dataset: CIFAR-10

CIFAR-10 consists of 60,000 color images (32×32 pixels) across 10 classes: airplane, automobile, bird, cat, deer, dog, frog, horse, ship, and truck.

Table 1: Dataset Split

Split	Samples	Purpose
Training	45,000	Model training
Validation	5,000	Hyperparameter tuning
Test	10,000	Final evaluation

All images were normalized to the range $[0, 1]$ by dividing pixel values by 255.

3.2 Model Architectures

Both teacher and student models follow a similar convolutional architecture, differing primarily in the number of filters.

3.2.1 Teacher Model

Table 2: Teacher Model Architecture

Layer	Output Shape	Filters	Operations
Input	$32 \times 32 \times 3$	–	–
Conv Block 1	$16 \times 16 \times 128$	128	Conv2D, BN, ReLU, MaxPool
Conv Block 2	$8 \times 8 \times 256$	256	Conv2D, BN, ReLU, MaxPool
Global Avg Pool	256	–	GlobalAveragePooling2D
Dropout	256	–	Dropout (0.3)
Output	10	–	Dense (logits)

3.2.2 Student Model

Table 3: Student Model Architecture

Layer	Output Shape	Filters	Operations
Input	$32 \times 32 \times 3$	–	–
Conv Block 1	$16 \times 16 \times 64$	64	Conv2D, BN, ReLU, MaxPool
Conv Block 2	$8 \times 8 \times 128$	128	Conv2D, BN, ReLU, MaxPool
Global Avg Pool	128	–	GlobalAveragePooling2D
Dropout	128	–	Dropout (0.3)
Output	10	–	Dense (logits)

The student model uses half the number of filters compared to the teacher, resulting in approximately $4 \times$ fewer parameters in the convolutional layers.

3.3 Distillation Framework

The custom DistillerKD class implements the knowledge distillation training loop:

Listing 1: Core Distillation Training Step

```
1 def train_step(self, data):
2     x, y = data
3     T = self.temperature
4
5     # Get teacher predictions (no gradient tracking)
6     t_logits = self.teacher(x, training=False)
7
8     with tf.GradientTape() as tape:
9         # Get student predictions
10        s_logits = self.student(x, training=True)
11
12        # Hard loss: cross-entropy with true labels
13        ce_val = self.ce(y, s_logits)
14
15        # Soft loss: KL divergence with temperature
16        t_soft = tf.nn.softmax(t_logits / T)
17        s_soft = tf.nn.softmax(s_logits / T)
18        kl_val = self.kld(t_soft, s_soft) * (T**2)
19
20        # Combined loss
21        loss = (1.0 - alpha) * ce_val + alpha * kl_val
22
23    # Update student weights
24    grads = tape.gradient(loss, self.student.trainable_variables)
25    self.optimizer.apply_gradients(zip(grads, ...))
```

3.4 Training Configuration

Table 4: Training Hyperparameters

Parameter	Teacher	Student (KD)
Optimizer	Adam	Adam
Learning Rate	0.001	0.001
Batch Size	128	128
Epochs	5	5
Temperature (T)	–	4.0
Alpha (α)	–	0.5

4 Results

4.1 Training Progress

4.1.1 Teacher Model Training

Table 5: Teacher Training History

Epoch	Train Acc	Train Loss	Val Acc	Val Loss
1	37.94%	1.7083	12.10%	3.1096
2	46.80%	1.4681	39.94%	1.7439
3	49.91%	1.3950	43.16%	1.5763
4	52.03%	1.3402	46.24%	1.4719
5	53.70%	1.2975	50.44%	1.4124

4.1.2 Student Model Training (with KD)

Table 6: Student Training History

Epoch	Train Acc	Hard CE	Soft $KL \times T^2$	Val Acc
1	34.11%	1.7111	0.3419	11.84%
2	43.21%	1.3493	0.3497	41.18%
3	45.62%	1.6107	0.2340	33.72%
4	47.14%	1.4738	0.2940	33.64%
5	48.13%	1.4454	0.2264	51.26%

4.2 Final Test Performance

Table 7: Test Set Results

Model	Test Accuracy
Teacher	48.64%
Student (with KD)	50.58%

Key Finding: The distilled student model outperformed the teacher by approximately 2 percentage points, demonstrating successful knowledge transfer.

4.3 Loss Component Analysis

The final test set loss breakdown for the student model:

Table 8: Test Loss Components

Component	Value
Hard Cross-Entropy ($\mathcal{L}_{CE}$)	1.4013
Soft KL Divergence ($\mathcal{L}_{KL}$)	0.0206
Total KD Loss ($\mathcal{L}_{KD}$)	0.8652

The low KL divergence (0.0206) indicates that the student has successfully learned to mimic the teacher’s probability distributions.

5 Discussion

5.1 Why Did the Student Outperform the Teacher?

Several factors may explain this counterintuitive result:

1. **Regularization Effect:** The soft labels from the teacher act as a form of label smoothing, which can reduce overfitting and improve generalization.
2. **Dark Knowledge Transfer:** The student benefits from learning inter-class relationships that are not available in hard labels alone.
3. **Model Capacity:** The smaller student model may be better suited to the limited training data (5 epochs), avoiding the overfitting that larger models are prone to.

5.2 Role of Temperature

The temperature $T = 4.0$ was chosen to produce sufficiently soft distributions. At this temperature, the teacher’s probability distributions reveal more information about class similarities, making it easier for the student to learn nuanced patterns.

5.3 Balancing Hard and Soft Losses

With $\alpha = 0.5$, equal weight was given to learning from true labels and from the teacher’s soft predictions. This balance allows the student to:

- Maintain accuracy on the actual classification task (via hard loss)
- Learn generalized patterns from the teacher (via soft loss)

6 Conclusion

This report demonstrated the implementation and effectiveness of knowledge distillation for neural network compression. Key takeaways include:

1. Knowledge distillation successfully transfers knowledge from a larger teacher model to a smaller student model.
2. The distilled student achieved 50.58% test accuracy, outperforming the teacher’s 48.64% accuracy despite having fewer parameters.
3. Temperature scaling ($T = 4.0$) effectively softens the teacher’s probability distributions to reveal inter-class relationships.
4. The combination of hard labels and soft distillation loss provides a robust training signal for the student.

6.1 Future Work

Potential improvements include:

- Training for more epochs with early stopping
- Experimenting with different temperature values
- Using data augmentation to improve both teacher and student
- Exploring intermediate layer matching (feature-based distillation)
- Comparing with a student trained without distillation (baseline)

References

1. Hinton, G., Vinyals, O., & Dean, J. (2015). *Distilling the Knowledge in a Neural Network*. arXiv preprint arXiv:1503.02531.
2. Krizhevsky, A. (2009). *Learning Multiple Layers of Features from Tiny Images*. Technical Report, University of Toronto.
3. TensorFlow Documentation. *Knowledge Distillation*. <https://www.tensorflow.org/>

A Knowledge Distillation Workflow

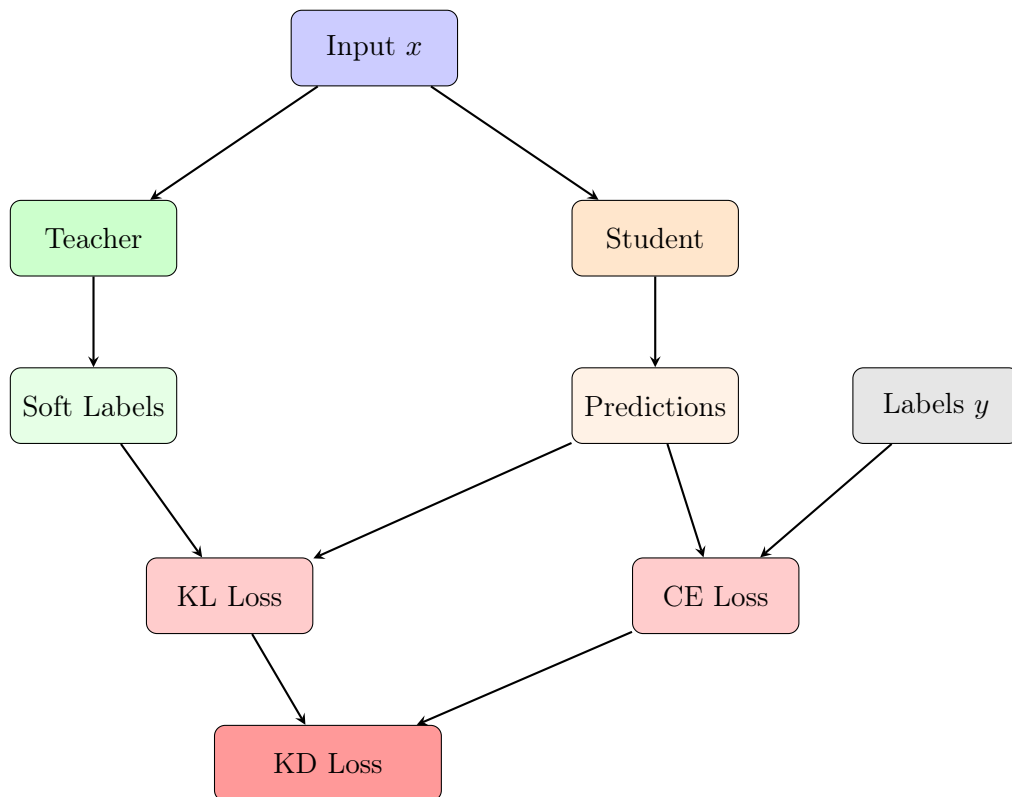


Figure 1: Knowledge Distillation Workflow: The student learns from both the teacher’s soft labels (KL Loss) and the ground truth labels (CE Loss).